Lab : Cloud Deployment Simulation

Cloud Computing - GCYS2

ENSAT 2025/2026

Zakariae Jabiri

Mohamed Nour Hamdane

Contents

1 Project Overview

1.1 Goal

The goal of this lab is to simulate a public cloud deployment using a local virtualized environment. Instead of a real cloud provider, the infrastructure is built on virtual machines that replicate the architecture of a production cloud setup.

The lab covers the core building blocks of modern cloud infrastructure: reverse proxying, HTTPS encryption, load balancing, and firewall configuration — ensuring backend servers are only reachable through the proxy.

Finally, a monitoring stack using Prometheus and Grafana is integrated to observe system metrics in real time, validated through a stress test using ApacheBench.

1.2 Architecture

The infrastructure consists of 4 virtual machines interconnected through a private network (`10.10.10.0/24`). The `nginx_proxy` machine is the only one exposed to the outside, acting as the single entry point for all incoming requests. It handles reverse proxying, load balancing, TLS termination, and firewall rules. The two web servers (`web_server_1` and `web_server_2`) are isolated on the private network and only reachable through the proxy. A dedicated `monitoring` VM runs Prometheus and Grafana to collect and visualize metrics from all machines.

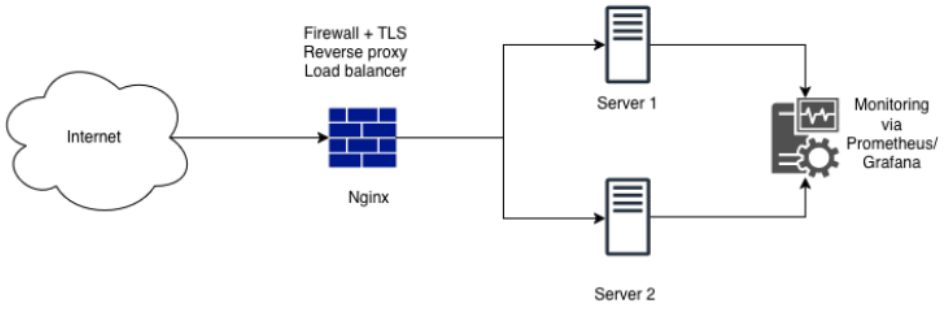

Figure 1: Overall architecture of the simulated cloud deployment

2 Lab Requirements

2.1 Virtual Machines

VM Name	Private IP	Role
nginx_proxy	10.10.10.1	Reverse proxy, Load balancer, Firewall
web_server_1	10.10.10.2	Flask web application
web_server_2	10.10.10.3	Flask web application
monitoring	10.10.10.4	Prometheus + Grafana

Table 1: Virtual Machines Summary

2.2 Network Configuration

The `nginx_proxy` machine is configured with two network interfaces: a NAT adapter used for outbound connectivity to external networks (serving as the public-facing interface), and a second interface connected to the internal network `10.10.10.0/24`. The web servers and the monitoring VM each have a single interface connected to the private network only, making them completely isolated from the outside.

IP addresses were manually assigned on the private network interfaces as shown in Table 1.

2.3 Tools and Software

Tool	Installed On	Purpose
Nginx	nginx_proxy	Reverse proxy, load balancer
OpenSSL	nginx_proxy	Self-signed certificate generation
UFW	All VMs	Firewall rules
Flask	web_server_1, web_server_2	Web application
Prometheus	monitoring	Metrics scraping and storage
Node Exporter	All VMs	Exposes system metrics on port 9100
Grafana	monitoring	Metrics visualization
ApacheBench	nginx_proxy	Stress testing

Table 2: Tools and Software Summary

3 Web Application Setup

A simple web application was developed using Flask (Python) and deployed on both `web_server_1` and `web_server_2`, each running on port `5000`. The application exposes a single `GET /` route that returns a plain text response identifying the server, allowing us to verify that requests are correctly distributed between the two backends.

Listing 1: Flask app (web_server_1)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def index():
    return "Hello from server_1"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

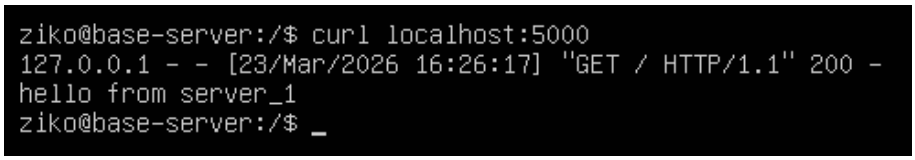

Figure 2: curl localhost:5000 on web_server_1

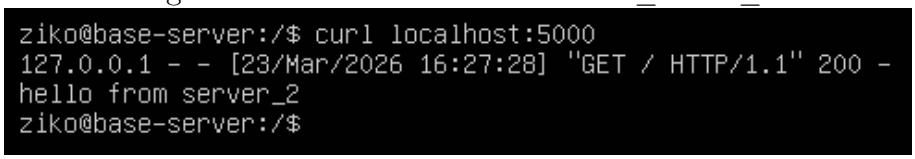

Figure 3: curl localhost:5000 on web_server_2

4 Reverse Proxy

4.1 Concept

A reverse proxy is a server positioned in front of backend servers that intercepts all incoming client requests and forwards them to the appropriate backend. Unlike a forward proxy which acts on behalf of the client, a reverse proxy acts on behalf of the servers — the client never communicates directly with the backend.

In this lab, `nginx` is used as the reverse proxy on `nginx_proxy`. It receives all HTTP/HTTPS requests from the host machine and forwards them to the web servers on the private network.

4.2 Nginx Configuration

The reverse proxy, HTTPS, and load balancer are all configured in a single file

`/etc/nginx/sites-enabled/webapp.conf`.The `proxy_pass` directive forwards all incoming requests to the upstream backend group. The full configuration is shown in Section 6.3.

5 Firewall Rules

5.1 Concept

UFW (Uncomplicated Firewall) is a user-friendly interface for managing `iptables` rules on Linux. It allows administrators to easily allow or deny incoming and outgoing traffic on specific ports. In this lab, UFW is configured on all VMs to restrict access and ensure that each machine is only reachable through the intended channels.

5.2 nginx_proxy Rules

The proxy is the only machine exposed to the outside, so it allows HTTP, HTTPS, and SSH. Port 9100 is allowed from the monitoring VM for node exporter scraping.

5.3 Web Servers Rules

The web servers only accept SSH from anywhere, application traffic on port 5000 from the proxy only, and node exporter traffic on port 9100 from the monitoring VM only. This ensures the backend is never directly accessible from the outside.

VM	Port	Allowed From
nginx_proxy	22, 80, 443, 9100	Anywhere / monitoring
web_server_1	22	Anywhere
	5000	10.10.10.1 (proxy)
	9100	10.10.10.4 (monitoring)
web_server_2	22	Anywhere
	5000	10.10.10.1 (proxy)
	9100	10.10.10.4 (monitoring)
monitoring	22, 3000, 9090, 9100	Anywhere

Table 3: UFW Firewall Rules Summary

6 HTTPS - Self-Signed Certificate

6.1 Concept

A self-signed certificate is a TLS certificate that is signed by the same entity that created it, rather than by a trusted Certificate Authority (CA) such as Let's Encrypt. While it provides encryption, it does not offer trust verification — browsers will show a warning when accessing a site using a self-signed certificate. In this lab, a self-signed certificate is sufficient since the goal is to enable HTTPS in a local simulated environment.

6.2 Certificate Generation

The certificate was generated on `nginx_proxy` using OpenSSL and stored in `/etc/nginx/ssl/`:

Listing 2: Generating a self-signed certificate

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/nginx/ssl/nginx.key \
    -out /etc/nginx/ssl/nginx.crt
```

6.3 Nginx HTTPS Configuration

The HTTPS configuration is handled in `webapp.conf`. HTTP traffic on port 80 is automatically redirected to HTTPS via a 301 redirect. The SSL block on port 443 handles the TLS handshake using the generated certificate and forwards requests to the backend.

```
ziko@base-server:/etc/nginx$ cat /etc/nginx/sites-enabled/webapp.conf
upstream backend{
        server 10.10.10.2:5000;
        server 10.10.10.3:5000;
}

server{
        listen 80;
        server_name _;
        return 301 https://$host$request_uri;
}

server{
        listen 443 ssl;
        server_name _;

        ssl_certificate /etc/nginx/ssl/nginx.crt;
        ssl_certificate_key /etc/nginx/ssl/nginx.key;

        location / {
                proxy_pass http://backend;
                proxy_set_header Host $host;
                proxy_set_header X-Real-IP $remote_addr;
        }
}
```

Figure 4: webapp.conf configuration

7 Load Balancer

7.1 Concept

Load balancing is the process of distributing incoming requests across multiple backend servers to optimize performance, avoid overloading a single server, and improve availability. If one server goes down, the load balancer redirects traffic to the remaining ones automatically.

Nginx supports several load balancing strategies: Round Robin (default), Least Connections, IP Hash, Random, and Weighted. These strategies can also be combined.

7.2 Round Robin Strategy

In this lab, the Round Robin strategy is used. Requests are distributed sequentially across the backend servers — the first request goes to `web_server_1`, the second to `web_server_2`, the third back to `web_server_1`, and so on.

7.3 Nginx Upstream Configuration

The load balancer is configured using the `upstream` block in `webapp.conf`, which declares both backend servers. The `proxy_pass` directive then points to this upstream group. The full configuration is shown in Section 6.3.

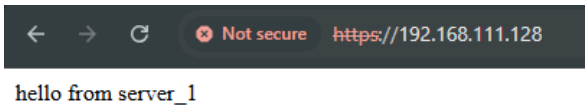

Figure 5: Request served by web_server_1

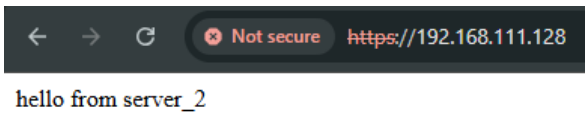

Figure 6: Request served by web_server_2

8 App Metrics Monitoring

8.1 Prometheus

8.1.1 Concept

Prometheus is an open-source monitoring engine that periodically scrapes metrics from declared targets through HTTP endpoints. It stores the collected data in a time-series database and provides a query language (PromQL) to retrieve custom metrics. In this lab, Prometheus runs on the `monitoring` VM and scrapes system metrics from all four machines via `node_exporter`.

8.1.2 Node Exporter

Node Exporter is a Prometheus agent installed on each VM. It exposes system-level metrics such as CPU usage, memory consumption, disk I/O, and network traffic on port `9100` at the endpoint `/metrics`.

8.1.3 Configuration

Prometheus is configured via `prometheus.yml`, where all four targets are declared:

```
ziko@monitoring:~/prometheus-3.1.0.linux-amd64$ cat prometheus.yml
# Global directive
global:
  # The scrape interval
  scrape_interval: 15s

scrape_configs:

  # Declare the job name
  - job_name: node_webapp_backend
    static_configs:
      - targets:
        - 10.10.10.2:9100
        - 10.10.10.3:9100
        - 10.10.10.1:9100
        - localhost:9100
ziko@monitoring:~/prometheus-3.1.0.linux-amd64$
```

Figure 7: prometheus.yml configuration

8.1.4 Targets Status

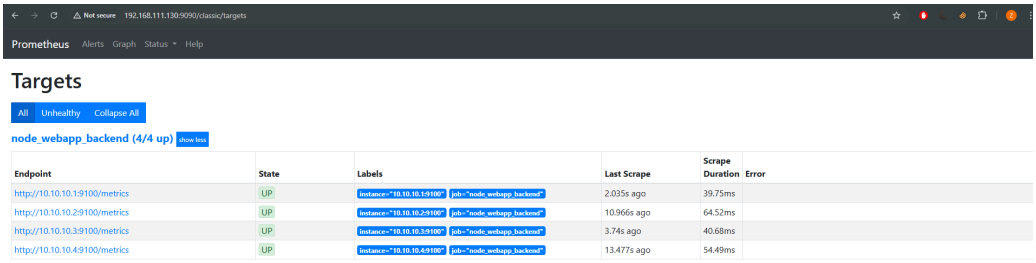

Figure 8: Prometheus targets — all 4 instances UP

8.2 Grafana

8.2.1 Concept

Grafana is an open-source data visualization tool that sits on top of Prometheus and provides interactive dashboards. It does not collect data itself — it queries Prometheus using PromQL and displays the results as graphs, charts, and gauges in real time.

Due to connectivity and routing constraints within the virtual lab, Grafana was ultimately accessed directly via the monitoring server's NAT IP address (http://192.168.111.130:3000), instead of being proxied through Nginx. This ensured reliable access to the dashboard while maintaining internal monitoring functionality.

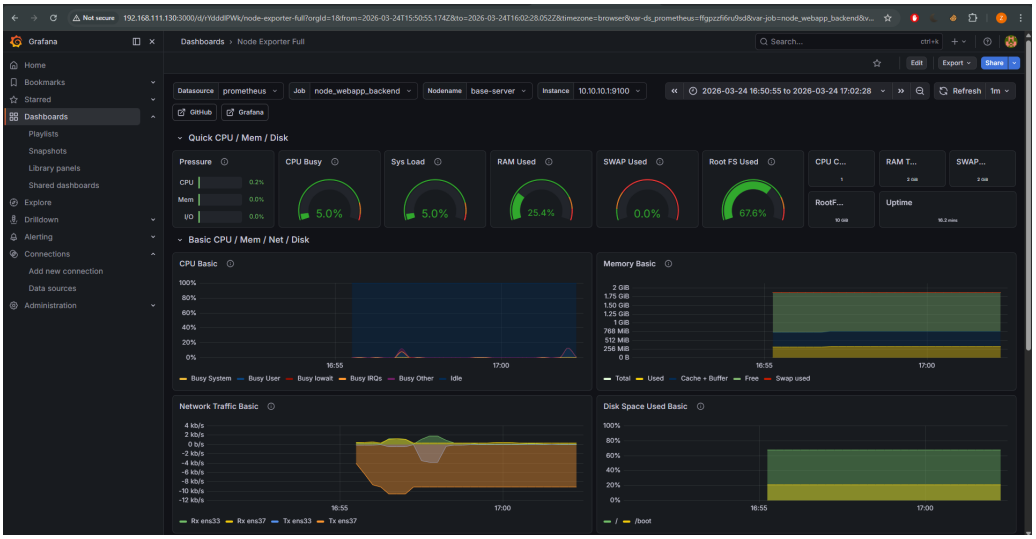

Figure 9: Node Exporter Full dashboard in Grafana

8.3 Stress Test

8.3.1 ApacheBench Results

A stress test was performed using ApacheBench from `nginx_proxy`, sending 500 requests with a concurrency level of 10 to `https://192.168.111.128/`:

Listing 3: ApacheBench command

```
ab -n 500 -c 10 https://192.168.111.128/
```

```
Server Port:            443
SSL/TLS Protocol:       TLSv1.2,ECDHE-RSA-AES256-GCM-SHA384,2048,256
Server Temp Key:        X25519 253 bits

Document Path:          /
Document Length:        166 bytes

Concurrency Level:      10
Time taken for tests:   0.516 seconds
Complete requests:      500
Failed requests:        0
Non-2xx responses:      500
Total transferred:      163500 bytes
HTML transferred:       83000 bytes
Requests per second:    969.69 [#/sec] (mean)
Time per request:       10.313 [ms] (mean)
Time per request:       1.031 [ms] (mean, across all concurrent requests)
Transfer rate:          309.66 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        1    6   2.6      6      10
Processing:     1    5   2.6      5       9
Waiting:        1    4   2.6      4       9
Total:          3   10   0.5     10      11

Percentage of the requests served within a certain time (ms)
  50%     10
  66%     10
  75%     10
  80%     11
  90%     11
  95%     11
  98%     11
  99%     11
 100%     11 (longest request)
ziko@base-server:~$ _
```

Figure 10: ApacheBench results — 500 requests, 0 failed, 969.69 req/s

8.3.2 Grafana Observations

During the stress test, the Grafana dashboard showed a clear spike in CPU usage on `nginx_proxy`, confirming that the monitoring stack is capturing real-time load on the infrastructure.

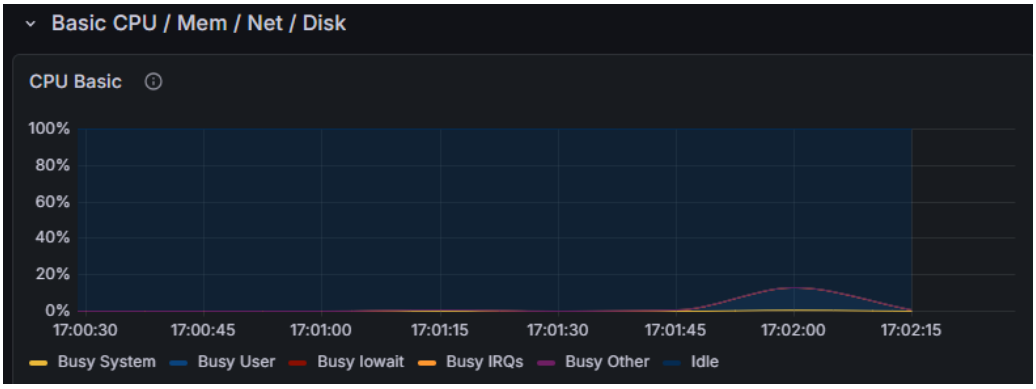

Figure 11: CPU spike observed on Grafana during stress test

9 Problems Encountered

9.1 Leftover IPTables Rules

After disabling UFW with `ufw disable`, Prometheus was still unable to scrape metrics from `web_server_1` and `web_server_2` on port 9100. The targets were showing *context deadline exceeded* errors despite the firewall being reported as inactive.

The cause was that UFW does not clean up the `iptables` rules it previously generated when disabled — the rules remain active in the kernel even after UFW is turned off, silently blocking the traffic.

The fix was to manually flush all iptables rules on both web servers:

Listing 4: Flushing iptables rules

```
sudo iptables -F
sudo iptables -X
sudo iptables -P INPUT ACCEPT
sudo iptables -P FORWARD ACCEPT
sudo iptables -P OUTPUT ACCEPT
```

After flushing, all 4 Prometheus targets immediately came back UP.

9.2 Grafana Origin Not Allowed

When accessing Grafana through the nginx proxy at `http://192.168.111.128:3000`, the browser displayed an *origin not allowed* error, preventing the Grafana UI from loading correctly.

The cause was that Grafana was being accessed via a different host than it was configured for — routing through the nginx proxy changed the origin of the request, which Grafana rejected as a security measure.

The fix was to add a NAT adapter directly to the `monitoring` VM, allowing direct access to Grafana via the monitoring VM's own IP, bypassing the proxy entirely.

10 Conclusion

This lab successfully simulated a public cloud deployment using a local virtualized environment. Starting from four bare virtual machines, a fully functional cloud infrastructure was built and configured step by step.

The `nginx_proxy` machine was set up as the single public entry point, handling reverse proxying, HTTPS encryption using a self-signed certificate, load balancing between two backend servers using the Round Robin strategy, and firewall rules to isolate the private network.

The two web servers were secured behind the proxy, only reachable through controlled ports, and a dedicated monitoring VM running Prometheus and Grafana provided real-time visibility into the health and performance of the entire infrastructure.

The stress test confirmed that the setup handles load correctly, with metrics captured live on the Grafana dashboard showing CPU and network spikes during the test.

Overall, this lab provided hands-on experience with the core building blocks of modern cloud infrastructure, from networking and security to monitoring and observability.